

Отчёт по лабораторной работе №4

Имитационное моделирование

Арбатова Варвара Петровна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	1. Агентное моделирование: от глобальных уравнений к локальным взаимодействиям	7
3.2	2. Основные компоненты агентной модели	8
3.3	3. Преимущества агентного подхода перед классическими моделями	9
3.4	4. Модель SIR в агентном подходе	10
4	Выполнение лабораторной работы	12
4.1	Базовый эксперимент	12
4.2	Сканирование коэффициента заразности	15
4.3	Исследование эффекта миграции	21
4.4	Многокритериальная оптимизация параметров	26
4.5	Сводная визуализация результатов	30
5	Выводы	32
	Список литературы	33

Список иллюстраций

4.1	Пик эпидемии и скорость распространения и влияние смертности	15
4.2	Пороговое значение β , при котором возникает эпидемия . .	20
4.3	Эффект миграции	26
4.4	Оптимальные параметры	29
4.5	Сводная визуализация результатов	31

Список таблиц

1 Цель работы

Здесь приводится формулировка цели лабораторной работы. Формулировки цели для каждой лабораторной работы приведены в методических указаниях.

Цель данного шаблона — максимально упростить подготовку отчётов по лабораторным работам. Модифицируя данный шаблон, студенты смогут без труда подготовить отчёт по лабораторным работам, а также познакомиться с основными возможностями разметки Markdown.

2 Задание

— Создать рабочий каталог для кода. — Установить необходимые пакеты. — Выполнить предложенный код. — Преобразовать код в литературный стиль. — Сгенерировать из литературного кода: — чистый код; — jupyter notebook; — документацию в формате Quarto. — Выполнить код из jupyter notebook. — Интегрировать документацию в формате Quarto в отчёт. — Добавить в код в литературном стиле вычисление для набора параметров. — Сгенерировать из литературного кода с параметрами: — чистый код; — jupyter notebook; — документацию в формате Quarto. — Выполнить код из jupyter notebook с параметрами. — Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

3.1 1. Агентное моделирование: от глобальных уравнений к локальным взаимодействиям

Агентное моделирование (Agent-Based Modeling, ABM) представляет собой мощный метод исследования сложных систем, который кардинально отличается от классического подхода, основанного на дифференциальных уравнениях. Вместо того чтобы описывать систему с помощью глобальных переменных и уравнений, ABM моделирует поведение множества автономных сущностей — **агентов** — и правила их взаимодействия. Наблюдая за тем, как локальные действия агентов порождают коллективные паттерны, мы можем понять эмерджентное поведение системы, возникающее «снизу вверх».

Этот подход особенно ценен, когда система обладает следующими свойствами:

- * **Гетерогенность:** участники системы не идентичны, а обладают индивидуальными характеристиками (возраст, иммунитет, социальная активность).
- * **Нелокальность и сложные взаимодействия:** взаимодействия происходят не «со всеми», а с ближайшим окружением.
- * **Адаптивность:** участники могут менять свое поведение в ответ на изменения среды.
- * **Стохастичность:** процессы в системе имеют случайную природу, что критически важно для малых популяций.

3.2 2. Основные компоненты агентной модели

Любая агентная модель, включая ту, что будет разработана в данной работе, строится из трех ключевых компонентов:

3.2.1 2.1. Агенты

Агенты — это активные, автономные сущности, населяющие модель. Каждый агент обладает: * **Свойствами (атрибутами):** набором характеристик, определяющих его состояние. Например, в эпидемиологической модели это может быть статус здоровья (восприимчив, инфицирован, выздоровел) и количество дней с момента заражения. * **Правилами поведения:** набором действий, которые агент может выполнять на каждом шаге моделирования. Например, взаимодействовать с другими агентами, перемещаться в пространстве, менять свой статус.

3.2.2 2.2. Среда

Это пространство, в котором существуют и взаимодействуют агенты. Среда может быть: * **Дискретной:** например, клеточная сетка или граф, где узлы представляют собой отдельные локации (города, районы). * **Непрерывной:** двумерное или трехмерное пространство с координатами. В нашей работе мы будем использовать **пространство в виде графа**, где вершины символизируют города, а ребра — возможные пути миграции между ними.

3.2.3 2.3. Взаимодействия

Взаимодействия — это правила, которые определяют, как агенты влияют друг на друга и на среду. Они могут быть: * **Локальными:** агент

взаимодействует только с теми, кто находится в той же локации. *

Глобальными: все агенты взаимодействуют со всеми. * **Через среду:** например, инфицированные агенты повышают вероятность заражения для восприимчивых агентов в своей локации.

3.3 3. Преимущества агентного подхода перед классическими моделями

В рамках четвертой лабораторной работы мы реализуем эпидемиологическую модель SIR. Классическая модель SIR, описываемая системой обыкновенных дифференциальных уравнений, имеет ряд ограничений, которые успешно преодолеваются с помощью ABM.

Характеристика	Классическая модель SIR (ОДУ)	Агентная модель SIR (ABM)
Популяция	Однородная, все индивиды идентичны.	Гетерогенная, каждый индивид уникален.
Взаимодействие	Полное перемешивание (массовое действие).	Локальное взаимодействие в пространстве или социальной сети.
Детерминизм	Детерминированная динамика (один и тот же результат при одинаковых начальных условиях).	Стохастическая динамика, учитывающая случайные флуктуации (особенно важна на начальном этапе).

	Классическая модель	Агентная модель SIR
Характеристика	SIR (ОДУ)	(ABM)
Численность	Рассматривается как непрерывная величина.	Рассматривается как дискретное целое число людей.
Пространство	Отсутствует.	Присутствует, позволяет моделировать миграцию и пространственную изоляцию.

3.4 4. Модель SIR в агентном подходе

В данной работе мы перенесем классическую компартментальную модель SIR в парадигму агентного моделирования. Это позволит нам учесть пространственную структуру (несколько городов) и стохастичность процесса, а также изучить такие эффекты, как влияние миграции на скорость распространения эпидемии.

3.4.1 4.1. Агенты (население)

Каждый агент представляет собой человека и характеризуется двумя ключевыми свойствами: * `status`: текущее состояние агента, которое может быть одним из трех: * : S (Susceptible) — восприимчивый к инфекции. * : I (Infectious) — инфицированный и способный заражать других. * : R (Recovered) — выздоровевший и получивший иммунитет. * `days_infected`: целое число, указывающее, сколько дней агент уже болеет. Это свойство

используется для определения момента выздоровления или смерти.

3.4.2 4.2. Среда (пространство)

В качестве среды используется граф, где каждый узел представляет собой отдельную локацию (город). Такое представление позволяет:

- * Учитывать пространственную структуру популяции.
- * Моделировать миграцию агентов между локациями с заданной вероятностью.
- * Изучать, как распространение инфекции зависит от связности и интенсивности перемещений.

3.4.3 4.3. Динамика модели

Процесс моделирования пошаговый, где каждый шаг соответствует одному дню. На каждом шаге для каждого агента выполняются следующие действия:

1. **Миграция:** с определенной вероятностью агент может переместиться в другую локацию (город). Вероятность миграции задается матрицей переходов.
2. **Передача инфекции:** если агент инфицирован, он может заразить восприимчивых агентов в своей текущей локации. Количество новых заражений определяется с помощью пуассоновского процесса, интенсивность которого зависит от того, выявлен ли агент (заразность β_{und}) или нет (β_{det}).
3. **Обновление счетчика болезни:** для инфицированных агентов увеличивается счетчик дней болезни (`days_infected`).
4. **Выздоровление или смерть:** если агент болеет дольше заданного периода (`infection_period`), он либо выздоравливает (переходит в состояние :R) с вероятностью $1 - death_rate$, либо умирает (удаляется из модели) с вероятностью `death_rate`.

4 Выполнение лабораторной работы

4.1 Базовый эксперимент

Запускает один эксперимент с фиксированными параметрами (по умолчанию) и сохраняет динамику численности агентов. Это служит для проверки работоспособности модели и получения базового понимания эпидемического процесса. График: `plots/sir_basic_dynamics.png` — четыре линии: $S(t)$, $I(t)$, $R(t)$ и общая численность (пунктир) (рис. 4.1). Позволяет визуально оценить пик эпидемии, скорость распространения и влияние смертности.

```
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots
using JLD2
include(scrdir("sir_model.jl"))
```

Параметры эксперимента

```
params = Dict{
  :Ns => [1000, 1000, 1000],
  :β_und => [0.5, 0.5, 0.5],
  :β_det => [0.05, 0.05, 0.05],
```

```

:infection_period => 14,
:detection_time => 7,
:death_rate => 0.02,
:reinfection_probability => 0.1,
:Is => [0, 0, 1],
:seed => 42,
:n_steps => 100,
)

```

Инициализация модели

```
model = initialize_sir(; params...)
```

Подготовка массивов для хранения данных

```

times = Int[]
S_vals = Int[]
I_vals = Int[]
R_vals = Int[]
total_vals = Int[]

```

Запуск симуляции вручную

```

for step = 1:params[:n_steps]
  Agents.step!(model, 1)
  push!(times, step)
  push!(S_vals, susceptible_count(model))
  push!(I_vals, infected_count(model))
  push!(R_vals, recovered_count(model))
  push!(total_vals, total_count(model))
end

```

Создаём DataFrame для удобства (опционально)

```
agent_df =  
DataFrame(time = times, susceptible = S_vals, infected = I_vals,  
recovered = R_vals)  
model_df = DataFrame(time = times, total = total_vals)
```

Визуализация

```
plot(  
agent_df.time,  
agent_df.susceptible,  
label = "S",  
xlabel = "time",  
ylabel = "S",  
)  
plot!(agent_df.time, agent_df.infected, label = "I")  
plot!(agent_df.time, agent_df.recovered, label = "R")  
plot!(agent_df.time, model_df.total, label = "Total (S + I + R)",  
linestyle = :dash)  
savefig(plotsdir("sir_basic_dynamics.png"))
```

Сохранение данных

```
@save datadir("sir_basic_agent.jld2") agent_df  
@save datadir("sir_basic_model.jld2") model_df
```

График:

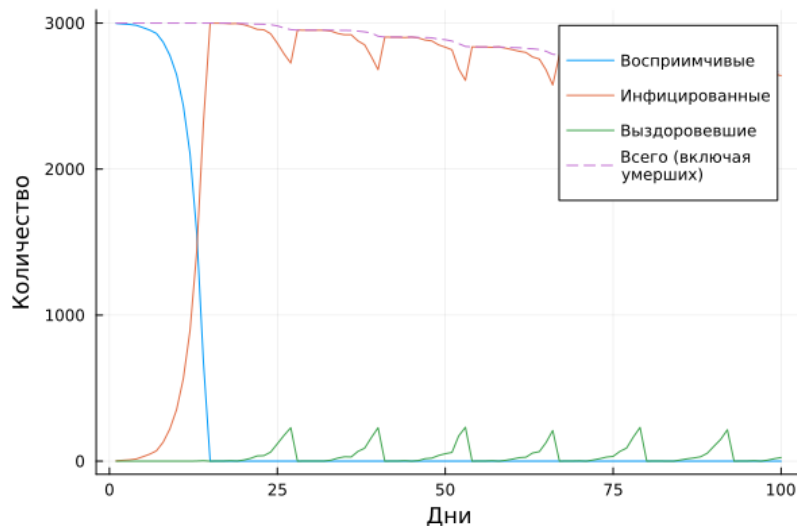


Рисунок 4.1: Пик эпидемии, скорость распространения и влияние смертности

4.2 Сканирование коэффициента заразности

Исследует, как изменение базовой заразности (β_{und} и пропорционально β_{det}) влияет на эпидемические показатели: пик заболеваемости, долю переболевших, число умерших. Выполняется параметрическое сканирование с несколькими повторными прогонами для учёта стохастичности. График: `plots/beta_scan.png` (рис. 4.2) показывает зависимость от β : — средняя пиковая доля инфицированных; — средняя конечная доля инфицированных; — средняя доля умерших (нормированная на численность). График позволяет найти пороговое значение β , при котором возникает эпидемия, и оценить, как увеличивается нагрузка на систему с ростом заразности

`scripts/scan_beta.jl`

```
using DrWatson
@quickactivate "project"
```

```
using Agents, DataFrames, Plots, CSV, Random
include(sourcedir("sir_model.jl"))
```

Функция для запуска одного эксперимента и возврата метрик Создаём β_{und} и β_{det} на основе скалярного beta Передаём в модель Ручной шаг (безопасный)

```
function run_experiment(p)
    beta = p[:beta]
     $\beta_{und}$  = fill(beta, 3)
     $\beta_{det}$  = fill(beta/10, 3)
    model = initialize_sir(;
        Ns = p[:Ns],
         $\beta_{und}$  =  $\beta_{und}$ ,
         $\beta_{det}$  =  $\beta_{det}$ ,
        infection_period = p[:infection_period],
        detection_time = p[:detection_time],
        death_rate = p[:death_rate],
        reinfection_probability = p[:reinfection_probability],
        Is = p[:Is],
        seed = p[:seed],
        n_steps = p[:n_steps], # XXXX XXXXXXXXXX XX XXXXXXXXXXXXXXX initialize_sir, I
    )
    infected_fraction(model) =
        count(a.status == :I for a in allagents(model)) /
        nagents(model)
    peak_infected = 0.0
    for step = 1:p[:n_steps]
        agent_ids = collect(allids(model))
```

```

for id in agent_ids
agent = try
model[id]
catch
nothing
end
if agent != nothing
sir_agent_step!(agent, model)
end
end
frac = infected_fraction(model)
if frac > peak_infected
peak_infected = frac
end
end
final_infected = infected_fraction(model)
final_recovered = count(a.status == :R for a in
allagents(model)) / nagents(model)
total_deaths = sum(p[:Ns]) - nagents(model)
return (
peak = peak_infected,
final_inf = final_infected,
final_rec = final_recovered,
deaths = total_deaths,
)
end

```

Диапазон значений beta (скаляр)

```
beta_range = 0.1:0.1:1.0  
seeds = [42, 43, 44]
```

Создаём список параметров

```
params_list = []  
for b in beta_range  
  for s in seeds  
    push!(  
      params_list,  
      Dict(  
        :beta => b, #   
        :Ns => [1000, 1000, 1000],  
        :infection_period => 14,  
        :detection_time => 7,  
        :death_rate => 0.02,  
        :reinfection_probability => 0.1,  
        :Is => [0, 0, 1],  
        :seed => s,  
        :n_steps => 100,  
      ),  
    )  
  end  
end
```

Запуск экспериментов

```
results = []  
for params in params_list  
  data = run_experiment(params)
```

```
push!(results, merge(params, Dict{pairs(data)}))
println("XXXXXXXX XXXXXXXXXXXX X beta = $(params[:beta]), seed =
$(params[:seed])")
end
```

Сохраняем все прогоны

```
df = DataFrame(results)
CSV.write(datadir("beta_scan_all.csv"), df)
```

Усреднение по повторам

```
using Statistics
grouped = combine(
groupby(df, [:beta]),
:peak => mean => :mean_peak,
:final_inf => mean => :mean_final_inf,
:deaths => mean => :mean_deaths,
)
```

График

```
plot(
grouped.beta,
grouped.mean_peak,
label = "XXX XXXXXXXX",
xlabel = "XXXXXXXXXXXX XXXXXXXXXX X",
ylabel = "XXX XXXXXXXXXXXXXXXX",
marker = :circle,
linewidth = 2,
```

```

)
plot!(
grouped.beta,
grouped.mean_final_inf,
label = "Максимальная доля инфицированных",
marker = :square,
)
plot!(grouped.beta, grouped.mean_deaths ./ 3000, label = "Средняя доля умерших",
marker = :diamond)
savefig(plotsdir("beta_scan.png"))
println("Записаны данные в файл data/beta_scan_all.csv в папке plots/beta_scan.png")

```

График:

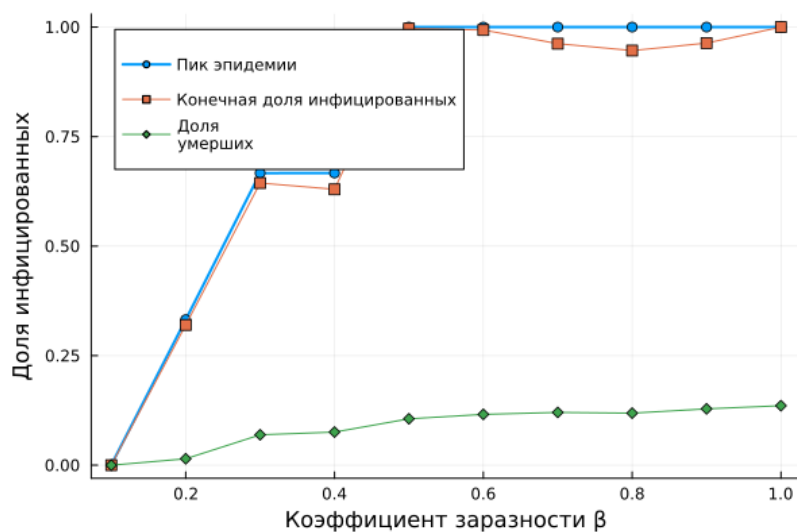


Рисунок 4.2: Пороговое значение beta, при котором возникает эпидемия

4.3 Исследование эффекта миграции

Исследует, как интенсивность перемещения людей между городами влияет на скорость распространения эпидемии (время достижения пика) и масштаб пика. Инфекция начинается только в одном городе, остальные изначально здоровы. График: `plots/migration_effect.png` (рис. 4.3) . Два подграфика (или один с двумя осями Y), показывающие: — время достижения пика (в днях) vs интенсивность миграции; — пиковую численность инфицированных vs интенсивность миграции. График демонстрирует, как ускорение обмена людьми между городами приводит к более раннему и более высокому пику эпидемии

```
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots, CSV, Random
include(sourcedir("sir_model.jl"))
```

Функция для создания матрицы миграции с заданной интенсивностью

```
function create_migration_matrix(C, intensity)
M = ones(C, C) .* intensity ./ (C-1)
for i = 1:C
M[i, i] = 1 - intensity
end
return M
end
```

Функция для измерения времени достижения пика Создаём матрицу миграции на основе интенсивности Ручной шаг

```

function peak_time(p)
migration_rates = create_migration_matrix(p[:C],
p[:migration_intensity])
model = initialize_sir(
Ns = p[:Ns],
⌀_und = p[:⌀_und],
⌀_det = p[:⌀_det],
infection_period = p[:infection_period],
detection_time = p[:detection_time],
death_rate = p[:death_rate],
reinfection_probability = p[:reinfection_probability],
Is = p[:Is],
seed = p[:seed],
migration_rates = migration_rates,
)
infected_frac(model) = count(a.status == :I for a in
allagents(model)) / nagents(model)
peak = 0.0
peak_step = 0
for step = 1:p[:n_steps]
agent_ids = collect(allids(model))
for id in agent_ids
agent = try
model[id]
catch
nothing
end
if agent != nothing

```

```

sir_agent_step!(agent, model)
end
end
frac = infected_frac(model)
if frac > peak
peak = frac
peak_step = step
end
end
return (peak_time = peak_step, peak_value = peak)
end

```

Сканирование интенсивности миграции

```

migration_intensities = 0.0:0.1:0.5
seeds = [42, 43, 44]
params_list = []
for mig in migration_intensities
for s in seeds
push!(
params_list,
Dict(
:migration_intensity => mig, # XXXXXX
:C => 3,
:Ns => [1000, 1000, 1000],
:β_und => [0.5, 0.5, 0.5],
:β_det => [0.05, 0.05, 0.05],
:infection_period => 14,
:detection_time => 7,

```

```

:death_rate => 0.02,
:reinfection_probability => 0.1,
:Is => [1, 0, 0],
:seed => s,
:n_steps => 150,
),
)
end
end

```

Запуск

```

results = []
for params in params_list
  data = peak_time(params)
  push!(results, merge(params, Dict{pairs(data)}))
  println(
    "XXXXXXXXXXXXXXXX migration_intensity =
$(params[:migration_intensity]), seed =
$(params[:seed])",
  )
end

```

Сохраняем все прогоны

```

df = DataFrame(results)
CSV.write(datadir("migration_scan_all.csv"), df)

```

Усреднение по повторам

```
using Statistics
grouped = combine(
  groupby(df, [:migration_intensity]),
  :peak_time => mean => :mean_peak_time,
  :peak_value => mean => :mean_peak_value,
)
```

Визуализация

```
plot(
  grouped.migration_intensity,
  grouped.mean_peak_time,
  marker = :circle,
  xlabel = "Среднее время миграции (сек)",
  ylabel = "Среднее значение пика (мВ)",
  label = "Среднее значение пика",
)
plot!(
  grouped.migration_intensity,
  grouped.mean_peak_value .* 3000,
  marker = :square,
  xlabel = "Среднее время миграции (сек)",
  ylabel = "Среднее значение пика (мВ)",
  label = "Среднее значение пика",
)
savefig(plotsdir("migration_effect.png"))
println("Среднее значение пика сохранено в data/migration_scan_all.csv в plots/migration_effect.png")
```

График:

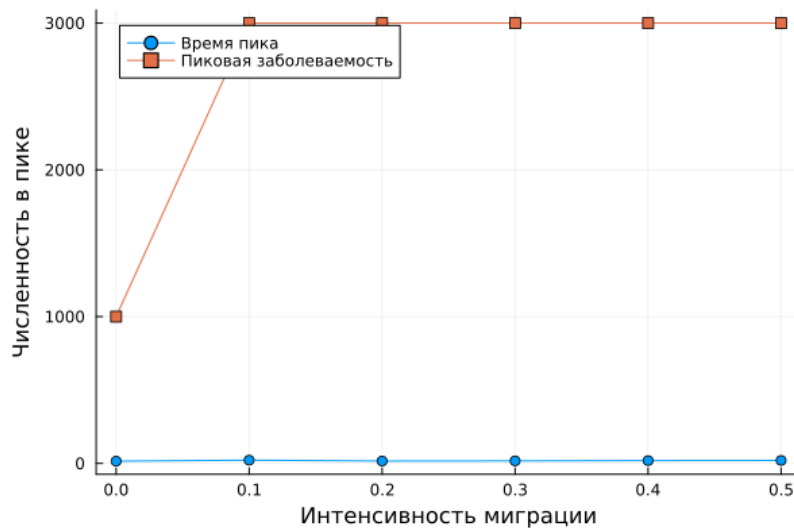


Рисунок 4.3: Эффект миграции

4.4 Многокритериальная оптимизация параметров

Ищет оптимальные комбинации параметров, минимизирующие одновременно два критерия: пиковую заболеваемость и долю умерших. Использует эволюционный алгоритм (Borg MOEA) из пакета BlackBoxOptim. Этот скрипт позволяет найти компромиссные стратегии управления (например, раннее выявление снижает пик, но не влияет на смертность). (рис. 4.4)

```
using DrWatson
@quickactivate "project"
using BlackBoxOptim, Random, Statistics
include(sourcedir("sir_model.jl"))
```

Целевая функция: минимизируем пиковую заболеваемость и смертность
 $x[1]$: β_{und} , $x[2]$: $death_rate$, $x[3]$: $detection_time$ Запускаем с несколькими повторами

```

function cost_multi(x)
model = initialize_sir(;
Ns = [1000, 1000, 1000],
x_und = fill(x[1], 3),
x_det = fill(x[1]/10, 3),
infection_period = 14,
detection_time = round(Int, x[2]),
death_rate = x[3],
reinfection_probability = 0.1,
Is = [0, 0, 1],
seed = 42,
n_steps = 100,
)
infected_frac(model) = count(a.status == :I for a in
allagents(model)) / nagents(model)
dead_count(model) = 3000 - nagents(model)
peak_infected = 0.0
replicates = 5
peak_vals = Float64[]
dead_vals = Int[]
for rep = 1:replicates
model = initialize_sir(;
Ns = [1000, 1000, 1000],
x_und = fill(x[1], 3),
x_det = fill(x[1]/10, 3),
infection_period = 14,
detection_time = round(Int, x[2]),
death_rate = x[3],

```

```

reinfection_probability = 0.1,
Is = [0, 0, 1],
seed = 42 + rep,
n_steps = 100,
)
for step = 1:100
Agents.step!(model, 1)
frac = infected_frac(model)
if frac > peak_infected
peak_infected = frac
end
end
push!(peak_vals, peak_infected)
push!(dead_vals, dead_count(model))
end
return (mean(peak_vals), mean(dead_vals) / 3000) # XXXX XXXXXXXX
end

```

Запуск оптимизации

```

result = bboptimize(
cost_multi,
Method = :borg_moea,
FitnessScheme = ParetoFitnessScheme{2}(is_minimizing = true),
SearchRange = [
(0.1, 1.0), # x_und
(3.0, 14.0), # detection_time
(0.01, 0.1), # death_rate
],

```

```

NumDimensions = 3,
MaxTime = 120, # 2
TraceMode = :compact,
)
best = best_candidate(result)
fitness = best_fitness(result)
println("XXXXXXXXXX XXXXXXXXX:")
println("β_und = $(best[1])")
println("XXXXX XXXXXXXXX = $(round{Int, best[2]}) XXXXX")
println("XXXXXXXXXX = $(best[3])")
println("XXXXXXXXXXXX XXXXXXXXX:")
println("XXX XXXXXXXXXXXXXXXX: $(fitness[1])")
println("XXXX XXXXXXX: $(fitness[2])")
save(datadir("optimization_result.jld2"), Dict("best" => best,
"fitness" => fitness))

```

```

Optimization stopped after 1 steps and 320.82 seconds
Termination reason: Max time (120.0 s) reached
Steps per second = 0.00
Function evals per second = 0.16
Improvements/step = NaN
Total function evaluations = 52

Best candidate found: [0.16987, 3.58283, 0.0595916]

Fitness: (0.00107, 0.00000) agg=0.00107

Оптимальные параметры:
β_und = 0.16986988343280895
Время выявления = 4 дней
Смертность = 0.05959155872843491
Достигнутые показатели:
Пик заболеваемости: 0.0010666666666666667
Доля умерших: 0.0

```

Рисунок 4.4: Оптимальные параметры

4.5 Сводная визуализация результатов

Этот скрипт загружает результаты параметрического сканирования (файл `data/beta_scan_all.csv`, созданный `scan_beta.jl`) и строит единый составной график, объединяющий три панели: — пик эпидемии и конечная доля инфицированных; — число умерших; — доля выздоровевших. Скрипт предназначен для итогового отчёта и не требует повторного запуска симуляций. График: `plots/comprehensive_analysis.png`. Три панели, позволяющие одновременно оценить: — порог возникновения эпидемии (рост пика); — нелинейное увеличение смертности; — насыщение доли переболевших. — В консоль может выводиться дополнительная статистика (например, значения β , при которых пик превышает 5%). В отличие от `scan_beta.jl`, этот скрипт не выполняет симуляции, а только визуализирует уже полученные данные. Это позволяет быстро перестраивать графики при изменении оформления, не тратя время на повторные расчёты. (рис. 4.5)

```
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots, CSV
include(sourcedir("sir_model.jl"))
```

Загружаем результаты сканирования

```
df = CSV.read(datadir("beta_scan_all.csv"), DataFrame)
```

Создаём составной график

```
p1 = plot(df.beta, df.peak, label = "P", xlabel = "β", ylabel = "P")
plot!(p1, df.beta, df.final_inf, label = "I")
p2 = plot(df.beta, df.deaths, xlabel = "β", ylabel = "D")
```

```

p3 = plot(df.beta, df.final_rec, xlabel = "β", ylabel = "Доля инфицированных")
plot(p1, p2, p3, layout = (3, 1), size = (800, 900))
savefig(plotsdir("comprehensive_analysis.png"))

```

График:

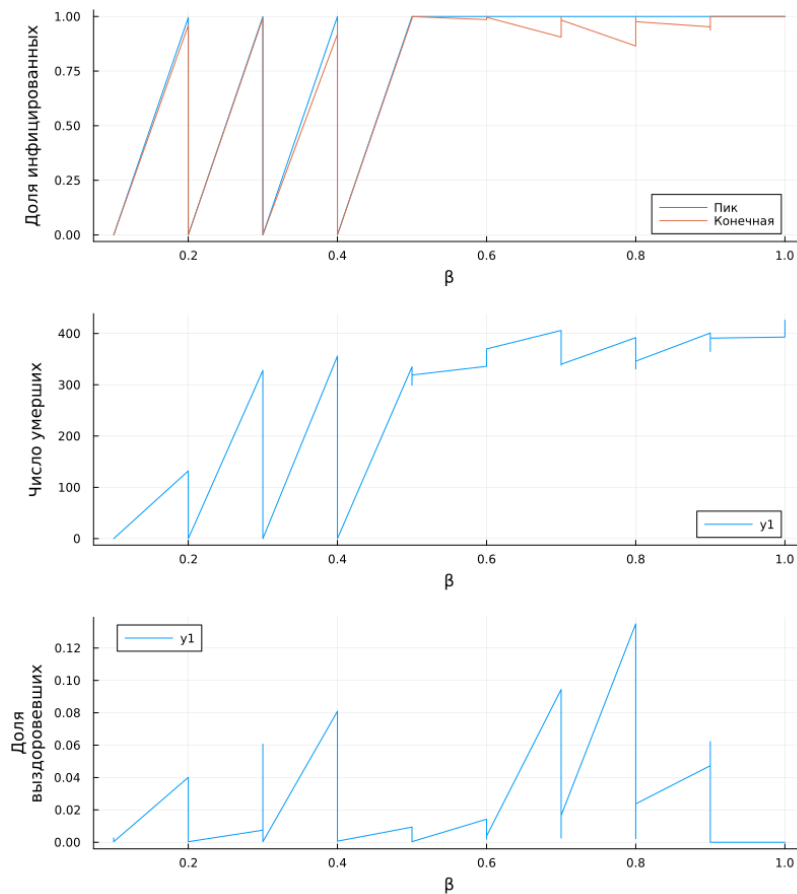


Рисунок 4.5: Сводная визуализация результатов

5 Выводы

Реализация модели SIR с помощью агентного подхода открывает возможности для более глубокого и гибкого анализа эпидемиологических процессов. Мы можем исследовать влияние индивидуальных характеристик, пространственной структуры и стохастичности на динамику эпидемии. Это делает ABM незаменимым инструментом для изучения сложных систем, где простота классических моделей уступает место реалистичности и детализации.

В ходе выполнения лабораторной работы вы создадите такую модель, проведете серию вычислительных экспериментов, проанализируете их результаты и научитесь оформлять их в виде воспроизводимых отчетов с использованием литературного программирования.

Список литературы

Лабораторная работ №4